

应龙的模型边界说明及平台接入规范

一、目的

为明确应龙平台、业务方、模型需求方在模型部署、试用、正式接入和后续维护中的职责边界，避免将业务试用、模型适配、效果验证等工作混入平台常规工作，制定本说明。

二、模型范围

模型类别	定义	具体模型	支持时间	应龙团队责任	业务方责任
主线模型	应龙平台统一规划和维护的公共模型能力	通用大模型：Qwen3.5 系列中小参数模型、Minimax 模型	已支持	负责部署、升级、监控和维护	申请调用 APIkey
		嵌入模型：Qwen3-Embedding-4B、Qwen3-VL-Embedding-2B	930 支持		
		重排模型：Qwen3-Reranker-4B、Qwen3-VL-Reranker-2B	930 支持		
非主线模型	超出主线模型列表，由具体业务或项目提出的开源模型、第三方模型、微调模型			提供规范、账号、权限、资源配额和平台能力	负责试用部署、正式自助部署、效果验证和模型维护

- 主线版本模型由应龙团队统一建设，并会持续更新。
- 非主线模型属于业务或项目专用能力，不默认纳入应龙团队实施范围。

三、总体原则

1. 应龙平台负责提供模型运行底座、统一调用、部署管理、权限管理、资源配额和基础监控能力。
2. 主线版本模型由应龙团队负责部署和维护，业务方按流程申请使用。

3. 非主线模型分为“试用部署”和“正式接入”两个阶段管理。
4. 试用部署阶段由业务方或模型需求方负责线下部署、模型启动、依赖适配、效果测试和资源评估。
5. 非主线模型试用通过后，如需正式接入平台，由模型需求方申请部署权限和资源配额，自行登录平台完成部署，并自行维护自己的模型服务。
6. 应龙团队不负责非主线模型的模型选型、权重获取、授权确认、代码改造、镜像制作、接口封装、效果验证和业务验收。

四、阶段边界

4.1 主线版本模型

主线版本模型由应龙团队负责部署、升级、监控和维护。

业务方需要使用时，提交调用申请，说明业务系统、调用场景、调用量。

4.2 非主线模型试用部署

非主线模型在尚未完成业务效果验证、资源规格评估、部署形态固化和正式接入材料准备前，属于试用部署阶段。

试用部署阶段：由**业务方**或**模型需求方**负责：

1. 获取模型权重、代码、授权文件和依赖材料。
2. 在线下环境，完成模型启动、依赖安装、GPU 适配、推理框架适配和接口暴露。
4. 完成业务样例测试、效果验证、参数调优和试用结论确认。
5. 完成资源评估，包括 GPU 型号、GPU 数量、并发、QPS 和延迟。

应龙团队在试用部署阶段可提供规范咨询、资源评估建议和正式接入条件说明，但不承担实际部署、适配调试、效果验证和试用交付责任。

4.3 非主线模型正式接入

非主线模型试用通过，并确认需要长期运行、统一调用或生产使用后，方可进入正式接入阶段。

正式接入阶段由应龙团队负责：

- 1. 提供平台接入规范、部署模板和材料清单。详见附件：[《应龙平台接入规范》](#)
- 3. 开通平台账号、部署权限、资源配额。

正式接入阶段由**业务方**或**模型需求方**负责：

- 1. 提交接入申请。
- 2. 自行准备模型权重、镜像、配置、启动文件、接口文档和测试样例。
- 3. 使用自有账号登录应龙平台，自行完成模型部署、启动、验证、发布和后续维护。
- 4. 负责模型效果、业务验收、版本升级、参数调整。

五、责任边界表

工作事项	应龙团队	业务方/模型需求方
主线模型部署维护	负责	使用
主线模型调用申请	审核并开通	发起申请
非主线模型选型	不负责	负责
非主线模型试用部署	不负责实施	负责
试用资源申请	审核资源合理性	发起申请并按期释放
非主线模型正式接入规范	提供	遵循
平台账号和部署权限	开通和管控	申请和使用
资源配额	审核和分配	提供评估依据

附件：《应龙平台模型接入规范》

一、目的

为规范模型服务接入应龙平台的流程，保障模型服务稳定、安全、可控运行，制定本规范。

二、基本原则

1. 应龙平台只纳管模型推理服务，不纳管带有业务流程处理逻辑的应用服务。
2. 平台提供统一模型接口、部署管理、调用管理和基础监控能力。
3. 平台不负责模型选型、适配、镜像制作、接口实现、性能测试、效果验证和合规自查。
4. 不符合准入要求的模型服务，平台可拒绝接入或要求整改。

三、不可接入范围

3.2 不接入范围

以下服务不属于平台模型服务纳管范围：

- 业务系统、业务中台、智能体应用、知识库应用。
- 包含业务规则判断、业务状态维护、业务数据读写或结果落库的应用服务。
- RAG 检索编排、Agent 任务规划、工具调用编排、知识库管理等业务应用层逻辑。

如业务场景需要业务流程处理、权限校验、知识检索、业务数据读写或结果落库，应由业务方在业务系统侧完成，再调用平台纳管的模型服务。

四、前置条件

模型服务接入时，业务方或模型需求方应准备好以下材料：

序号	材料名称	内容要求	是否必须
1	模型信息	模型名称、模型类型、模型版本、业务场景、接入责任方和联系人。	必须
2	模型权重	明确模型版本、来源、授权情况、存放路径、访问方式和校验方式。	必须
3	部署镜像	提供可拉取、可启动的 Docker 镜像，并明确镜像仓库、镜像版本和依赖说明。	必须
4	K8s 启动 YAML	用于部署模型服务的 YAML 文件，需符合本规范的 K8s 部署要求。	必须
5	接口文档	推荐 OpenAPI/Swagger 格式，包含接口路径、请求方法、请求参数、响应结构、错误码、超时设置和调用示例。	必须
6	性能测试报告	包含测试环境、GPU 型号、资源规格、测试数据、并发设置、QPS、吞吐量、平均延迟、P95/P99 延迟和资源占用情况。	必须
7	测试样例数据	提供不少于 10 条可复现测试数据，并说明预期响应或验证方式。	必须

五、基本准入要求

6.1 服务要求

要求项	规范说明
-----	------

服务边界	模型服务应仅提供模型推理能力，不应包含业务流程处理、业务状态维护、业务数据读写或结果落库逻辑。
接口能力	应提供清晰、稳定、可测试的推理接口，并与接口文档保持一致。
可部署性	镜像、配置、模型权重和 K8s 启动文件应能够在平台测试环境中复现部署。
可观测性	服务应提供健康检查接口或探针配置，便于平台进行基础监控。
可配置性	模型路径、端口、并发数、GPU 数量、batch_size 等可变参数应集中配置。

6.2 K8s 部署要求

接入责任方提交的 K8s 启动文件应满足以下要求：

要求项	规范说明
部署对象	应提供可复现部署的 Deployment、Service、ConfigMap、Secret 引用等必要资源。
端口声明	Service 和容器端口必须明确声明，并与接口文档一致。
模型挂载	模型权重挂载路径应清晰，Volume 名称建议统一为 model-path。
资源声明	必须声明 CPU、内存、GPU 等 requests 和 limits，GPU 资源名需与现场环境一致。
配置管理	GPU 数量、并发数、batch_size、模型路径、端口等可变参数应 集中配置 ，不得散落硬编码。
健康检查	必须提供 readinessProbe 和 livenessProbe；模型加载时间较长时，应提供 startupProbe。

镜像来源	镜像应来自平台可访问的可信仓库，并使用明确版本号，不建议使用 latest。
一键部署	原则上应支持一键部署；无法单一 YAML 部署的，应提供分步部署文档、依赖关系和验证方法。

七、K8s 部署示例

以下示例仅用于说明关键字段结构，实际部署文件应由接入责任方根据现场环境、镜像仓库、资源规格和平台要求调整。

```
# 1. 模型部署 (Deployment)
# =====
apiVersion: apps/v1
kind: Deployment
metadata:
  name: model-deployment
  namespace: default
  labels:
    app: model-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: model-deployment
  template:
    metadata:
      labels:
        app: model-deployment
    spec:
      containers:
        - name: model-container
          image: registry.company.com/models/my-model:v1.0.0 # 替换为实际部署镜像地址
          imagePullPolicy: IfNotPresent

# 【要求 1】明确定义并暴露在外的容器端口号
ports:
  - containerPort: 8080
    name: model-svc
```

```

# 启动命令示例(根据实际镜像进行调整)
command: ["/bin/bash", "-c"]
args:
  - "--model-dir"
  - "/mnt/models" # 容器内读取模型路径
  - "--config"
  - "/app/config/config.json" # 容器内读取参数配置路径
  - "--port"
  - "8080"

# GPU 资源限制(根据模型实际需要调整)
resources:
  limits:
    nvidia.com/gpu: "1" # 申请 1 张 GPU
  requests:
    nvidia.com/gpu: "1"

volumeMounts:
  # 【要求 2】挂载名称统一叫 "model-path"
  - name: model-path
    mountPath: /mnt/models # 容器内的挂载路径
    readOnly: true

  # 挂载参数配置文件
  - name: config-volume
    mountPath: /app/config

volumes:
  # 【要求 2】挂载名称统一叫 "model-path", 此处以 PVC(持久卷声明)为例
  # (实际部署时, 平台会根据该名称将真实的权重路径与容器进行绑定)
  - name: model-path
    persistentVolumeClaim:
      claimName: model-weight-pvc # 替换为实际的 PVC 名称, 或根据现场使用 HostPath

  # 挂载 ConfigMap 作为参数文件
  - name: config-volume
    configMap:
      name: model-configmap

```



```

        items:
          - key: config.json
            path: config.json

---
# =====
# 2. 服务暴露 (Service) - 使用 NodePort 方式
# =====
apiVersion: v1
kind: Service
metadata:
  name: model-svc
spec:
  # 使用 NodePort 类型, 方便外部客户端直接通过节点 IP 访问
  type: NodePort
  selector:
    app: model-deployment
  ports:
    - name: model-svc
      port: 8080          # Service 端口(集群内部访问端口)
      targetPort: 8080    # 容器端口(Deployment 中定义的 containerPort)
      nodePort: 30080     # 映射到物理机/宿主机上的端口(默认取值范围为 30000-32767), 可根据实际情况指定
      protocol: TCP

```